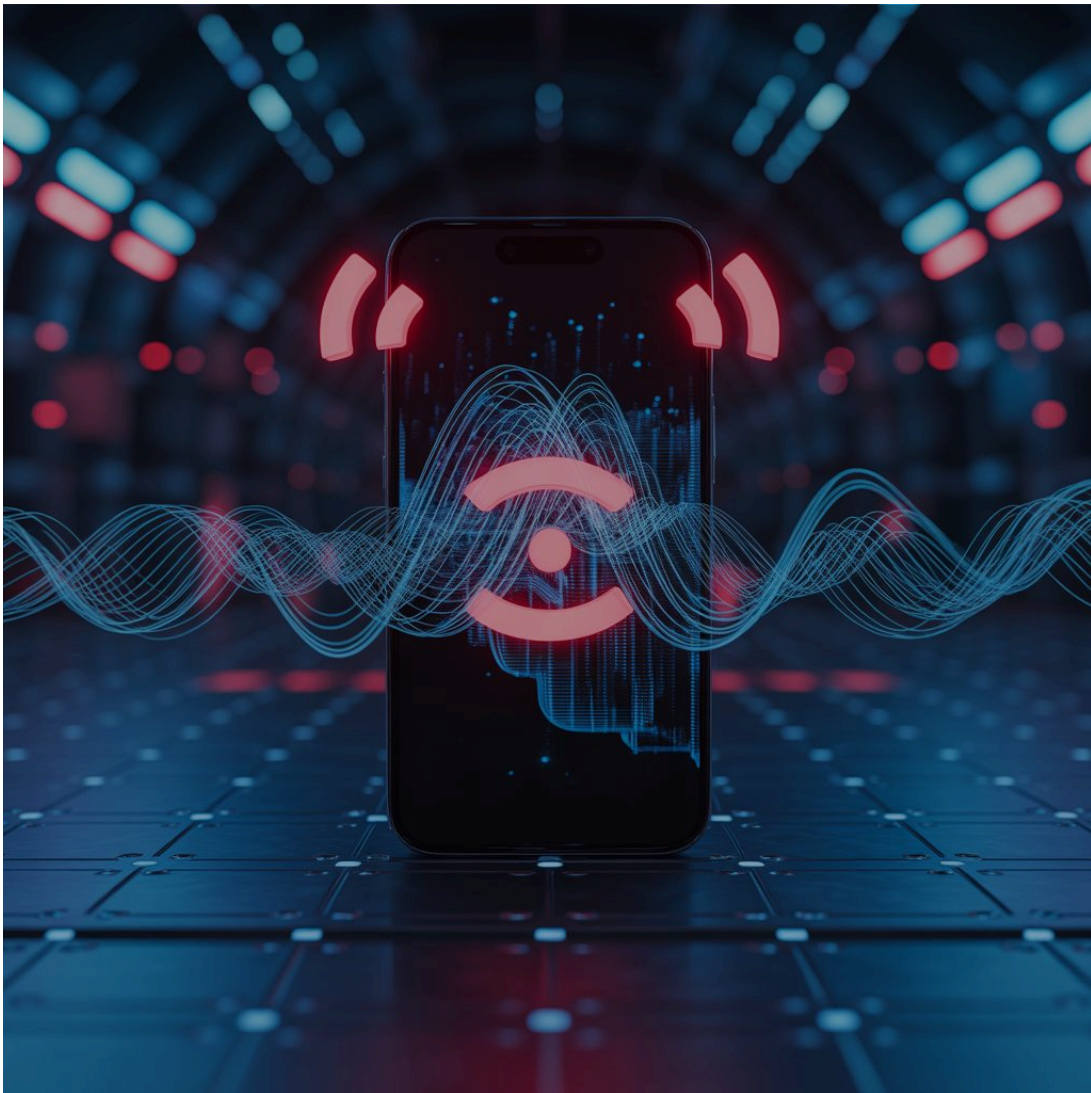


LTE to 2G Downgrade Attack Chain Implementation simulator Technical project report

Members:

- Brahmi Sarah Kaouther
- Madene Meriem
- Mostefaoui Manel



1. Introduction

This project is an educational, software only simulation that demonstrates the LTE-to-2G network downgrade attack (also known as a "2G IMSI Catcher" or "STINGRAY" style attack). The simulator visualizes, step-by-step, how a malicious actor can force a modern 4G LTE-capable mobile device to fall back onto the insecure 2G (GERAN) network, intercept its IMSI, and crack its weak A5/1 stream cipher encryption, all without physical hardware.

The project was built entirely in Python using Tkinter for the graphical interface and a custom publish/subscribe event bus for inter-module communication. It consists of two synchronized GUI windows: a dark-themed Attack Dashboard (the attacker's view) and a realistic Phone Mockup (the victim device's view).

2. Background & Attack Concept

2.1 The LTE → 2G Downgrade Attack

4G LTE is a secure, encrypted mobile standard. However, it is backward-compatible with legacy 2G/GSM networks. Attackers exploit this by:

- Setting up a rogue base station (IMSI catcher / "Stingray") that mimics a legitimate cell tower.
- Sending a fake RRC Connection Reject message to the victim's device, causing it to disconnect from LTE.
- The device automatically scans for available networks and connects to the attacker's rogue 2G BTS (Base Transceiver Station).
- Once on 2G, the phone is subject to A5/1 cipher encryption, which is cryptographically weak and can be cracked with known-plaintext attacks.
- The IMSI (International Mobile Subscriber Identity), a unique identifier for every SIM card, is transmitted in plaintext during 2G registration and can be captured.

2.2 Why This Is a Security Risk

A5/1 was introduced in the 1980s and has been publicly broken. Rainbow table attacks and known-plaintext attacks can recover the 64-bit session key in seconds to minutes. Once the attacker holds the IMSI and the session key, they can track the device, intercept calls and SMS, and impersonate the victim on the network. This simulation replicates the logical flow of the attack without any real radio hardware.

3. System Architecture

The project follows a decoupled, event-driven architecture. All modules communicate exclusively through a central publish/subscribe event bus. No module holds a direct reference to another, ensuring clean separation of concerns.

Module / File	Class / Component	Role / Purpose
<code>events.py</code>	Event Bus	Central pub/sub message broker connecting all modules
<code>ue_simulator.py</code>	UESimulator	Simulates the victim mobile device state machine
<code>attack_engine.py</code>	AttackEngine	Orchestrates the attack sequence step by step
<code>dashboard.py</code>	Dashboard	Dark-themed attacker GUI — live log + attack summary
<code>phone_mockup.py</code>	PhoneMockup	Realistic phone UI showing victim device status

3.1 Event Flow Diagram

The state machine transitions follow this sequence:



4. Module Code Explanations

4.1 events.py : The Event Bus

This is the backbone of the entire project. It implements a lightweight publish/subscribe (pub/sub) messaging pattern using a plain Python dictionary.

subscribe()	Registers a callback function to be called whenever the named event is published. If the event key does not exist in the <code>_listeners</code> dictionary, it is created first.
publish()	Iterates through all callbacks registered under the event name and calls each one with the provided data payload. If no listeners are registered, it silently does nothing.

Key design decision: Using a module-level dictionary (`_listeners`) makes the bus a global singleton, all modules that import it share the same instance without needing to pass it around explicitly.

4.2 ue_simulator.py : User Equipment (Phone) Simulator

UESimulator models the victim mobile device as a state machine with three sequential transitions. It runs on a background thread to avoid blocking the GUI event loop.

```
connect_to_lte() → sets state to LTE_CONNECTED, publishes state_change
receive_rrc_reject() → simulates the attacker's fake RRC Reject message
fallback_to_2g() → device searches and connects to rogue 2G BTS
run() → executes all three steps in sequence with delays
```

The `connect_to_lte()` method sets the internal state variable and fires a log event. The `receive_rrc_reject()` method simulates the critical exploit: the phone blindly accepts the unauthenticated reject message (a known vulnerability in 3GPP LTE standard up to Release 13). The `fallback_to_2g()` method simulates the phone selecting the attacker's rogue base station as the strongest available 2G signal.

4.3 attack_engine.py : The Attack Orchestrator

AttackEngine subscribes to the state_change event and activates once the device is confirmed on 2G. It then executes five attack phases sequentially, publishing log events after each step to drive the dashboard display.

- Step 1: Rogue BTS Confirmation: Confirms the phone is connected to the fake base station.
- Step 2: Cipher Detection: Detects that the 2G session is using A5/1 stream cipher.
- Step 3: A5/1 Key Cracking: Simulates a known-plaintext attack to recover the 64-bit session key (0xFA3C9B12E4D07A56 in the demo).
- Step 4: IMSI Capture: Captures the IMSI (603 01 0000000001) from the unprotected 2G registration channel.
- Step 5: Attack Complete: Fires the attack_complete event with the full payload (IMSI, key, cipher type), which triggers both GUIs to display the final summary.

4.4 dashboard.py : Attack Dashboard GUI

The Dashboard class builds the attacker-facing GUI using Tkinter. It has a dark (#0d0d0d) background with monospace Courier font throughout to evoke a terminal / hacking aesthetic.

Component	Description
Status Bar	Displays the current phone state (IDLE / LTE / DISCONNECTED / 2G) with color-coded labels. Updates via the state_change event.
Live Attack Log (ScrolledText)	A scrollable, read-only text widget. Each log entry is color-tagged: green=success, orange=warning, red=danger. New entries are appended and auto-scrolled.
Hint Label	A context-sensitive footer message that explains what is happening at each stage, e.g. 'Phone on 2G — attack running...'
Attack Summary Box	Hidden by default. Revealed only when attack_complete fires. Displays IMSI, session key, and cipher in a red-bordered ASCII art box.

4.5 phone_mockup.py : Victim Phone UI

PhoneMockup renders a stylized smartphone interface inside a Tkinter window, simulating how the victim's phone screen would look during the attack. It mirrors real mobile UI conventions: status bar with time, carrier name, signal bars, battery indicator, network badge, notification cards, system log, and a compromise summary panel.

- Signal Bars Canvas: A custom Tkinter Canvas draws 4 signal bars that fill green (LTE), orange (disconnecting), or red (2G/rogue).
- Network Badge Pill: Displays 'IDLE', 'LTE', '!', or '2G' in a colored rounded-rectangle badge.
- Carrier Label: Shows 'Network Operator' normally, but changes to 'ROGUE BASE STATION' in red once the downgrade occurs.
- Notification Cards: A stack of 3 notification slots that update with push-notification-style cards (LTE Connected, LTE Rejected, DOWNGRADED TO 2G, COMPROMISED).
- System Log: 4 scrolling label rows showing the most recent log messages with color-coding based on keywords (crack, IMSI, rogue → red; reject → orange).

- Device Compromised Panel: A pink (#fff0f0) frame revealed only when `attack_complete` fires, displaying the captured IMSI, cracked key, and encryption type.
- Clock: Auto-updates every 10 seconds using `root.after()` to display real system time.

5. Key Design Patterns

5.1 Publish / Subscribe Event Bus

All inter-module communication flows through `events.py`. This decouples the simulation engine from the GUI layer entirely. The `AttackEngine` never imports `Dashboard` or `PhoneMockup`, it simply publishes events, and any subscriber reacts independently. This also makes the system easily extensible: adding a new GUI window requires only subscribing to the existing events.

5.2 Thread Safety via `root.after()`

The `UESimulator` runs on a background thread (`threading.Thread`). All UI mutations in `PhoneMockup` are wrapped in `self.root.after(0, update)`, which schedules the update on the Tkinter main thread. This is the standard and correct approach to thread-safe Tkinter updates, preventing race conditions on widget state.

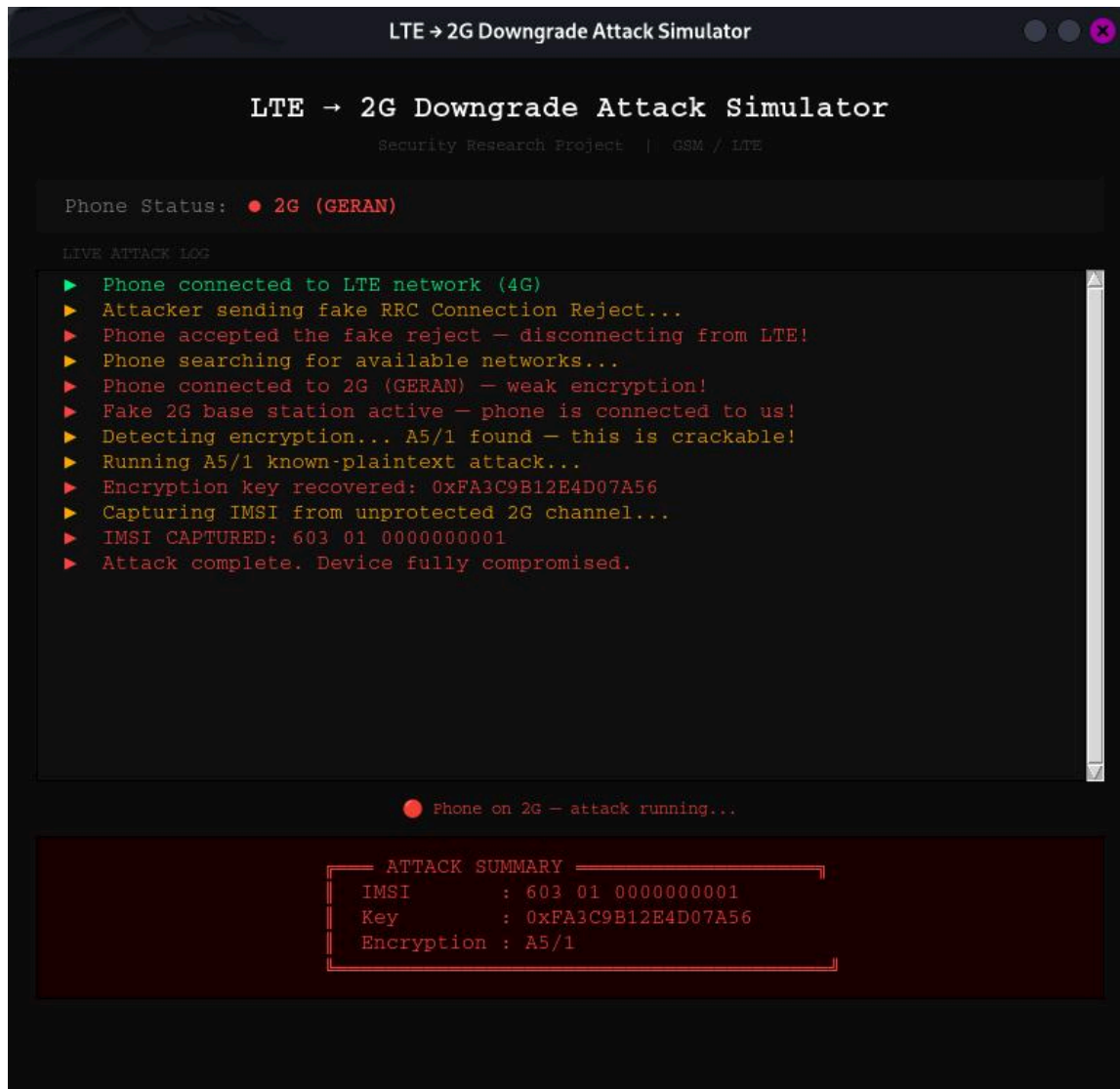
5.3 State Machine

The device lifecycle follows a strict state machine: `IDLE` → `LTE_CONNECTED` → `DISCONNECTED` → `2G_CONNECTED` → (`ATTACK_COMPLETE`). Each transition triggers a `state_change` event that all subscribers (`Dashboard`, `PhoneMockup`, `AttackEngine`) handle independently according to their own logic.

6. Application Output

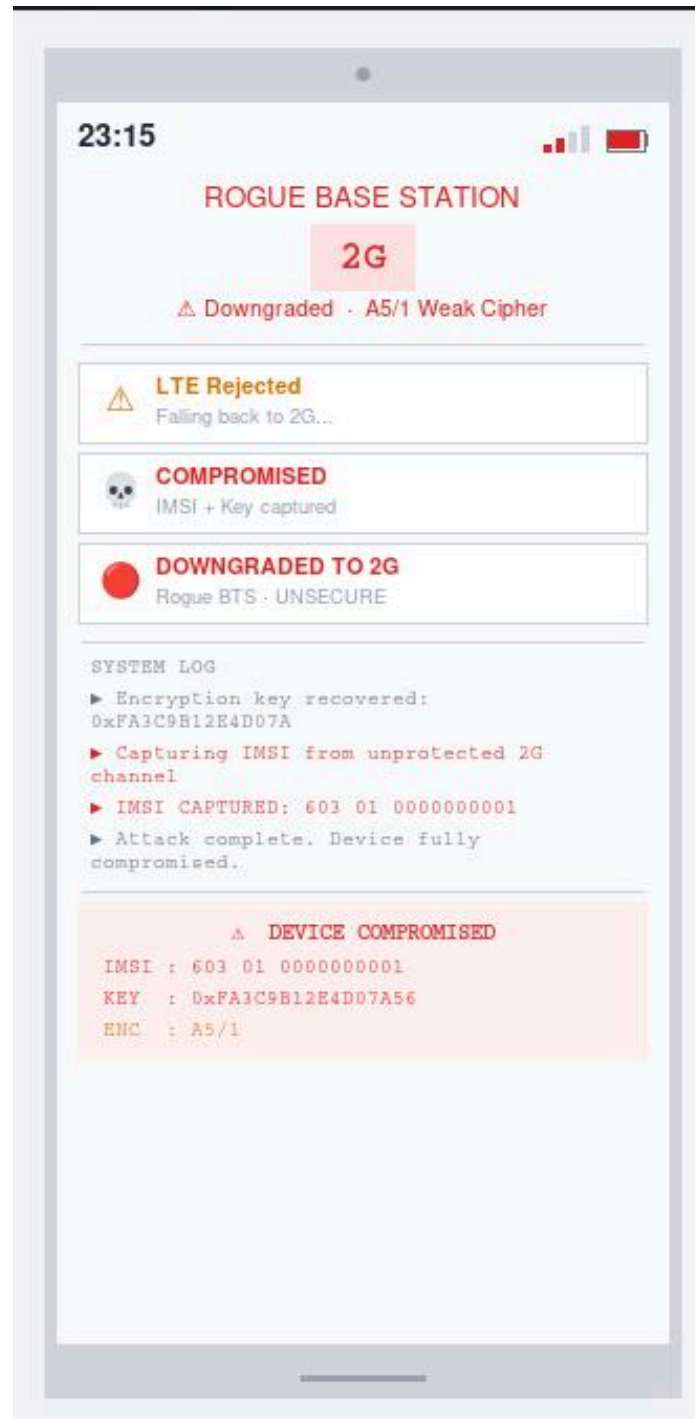
6.1 Attack Dashboard

The attacker-side dashboard (800×650px) shows the live attack log in real time, color-coded by severity level. The status bar at the top reflects the phone's current network state. Upon attack completion, the ASCII-art summary box is revealed at the bottom displaying the captured IMSI, recovered session key, and cipher used.



6.2 Phone Mockup

The phone mockup (320×620px) renders a realistic smartphone UI. It shows the carrier name change from 'Network Operator' to 'ROGUE BASE STATION', the signal bars dropping from full green (LTE) to partial red (2G), notification cards stacking up with the attack progression, and the red DEVICE COMPROMISED panel appearing at the end.



7. Full Attack Sequence Log

The following table documents every step logged during a full simulation run, matching the output visible in the dashboard screenshot:

#	Level	Log Message
1	SUCCESS	Phone connected to LTE network (4G)
2	WARNING	Attacker sending fake RRC Connection Reject...
3	DANGER	Phone accepted the fake reject – disconnecting from LTE!
4	WARNING	Phone searching for available networks...
5	DANGER	Phone connected to 2G (GERAN) – weak encryption!
6	DANGER	Fake 2G base station active – phone is connected to us!
7	WARNING	Detecting encryption... A5/1 found – this is crackable!
8	WARNING	Running A5/1 known-plaintext attack...
9	DANGER	Encryption key recovered: 0xFA3C9B12E4D07A56
10	WARNING	Capturing IMSI from unprotected 2G channel...
11	DANGER	IMSI CAPTURED: 603 01 0000000001
12	DANGER	Attack complete. Device fully compromised.

8. Conclusion

This project successfully demonstrates the complete LTE-to-2G downgrade attack lifecycle in a purely software-based, educational simulation. The event-driven Python architecture cleanly separates the simulation logic from the GUI layer, making the codebase extensible and well-structured.

The dual-window interface (Attack Dashboard + Phone Mockup) provides two simultaneous perspectives on the same attack: the attacker's operational view and the victim device's user experience, which makes the security implications immediately visible and intuitive.

From a security perspective, this simulation highlights a real and historically exploited vulnerability in mobile network standards. The A5/1 cipher used by 2G GSM networks is considered cryptographically broken, and the lack of mutual authentication in early LTE RRC procedures enabled downgrade attacks until 3GPP introduced countermeasures in later releases. Understanding these attack vectors is essential for mobile security research, network operator defense, and regulatory policy.